EXPENSEMATE

Final Software Project Report

Members: Areesha 24k-3043, Burhan 24k-3056

Technology	Supporting Tools	Architecture	Final Status
Python • SQLite • Streamlit	Pandas • Matplotlib • pytest	3-tier modular local client-server	152 tests passed • 97% coverage

1. Introduction & Objective

ExpenseMate is a standalone personal expense manager designed to run locally through a browser using Streamlit, with SQLite providing persistent storage. Its objective is to give a simple way to record income and expenses, organize transactions into categories, set monthly category budgets, monitor spending, view financial trends, and import/export transaction data. The system stores transactions, categories, and budgets in expensemate.db so data remains available after the application is closed and reopened. The project follows a modular three-tier structure separating presentation, business logic, and persistence.

- Primary objective: provide reliable personal budget tracking with clear feedback, validation, analytics, and persistent local data.
- Scope includes transaction management, custom categories, monthly budgets, CSV portability, financial summaries, and visual analytics.

2. Requirements Summary

The requirements baseline defines 16 functional requirements and 15 non-functional requirements. Core functional scope covers transaction CRUD, categories, monthly budgets, CSV import/export, analytics, savings calculation, financial summaries, and SQLite persistence. Quality requirements cover performance, usability, feedback, reliability, data integrity, CSV validation, modularity, automated testing, input/date validation, fault handling, resource usage, and data consistency.

Area	Implemented Requirement Summary
Transactions	Add, edit, delete and date-sort transactions; every transaction has one category.
Categories & Budgets	Custom categories plus monthly category limits, remaining-budget calculations and over-budget alerts.
CSV	Strict date,category,amount,type schema; validated import with all-or-nothing insertion; export support.
Analytics	Budget/spent/remaining charts, monthly income-vs-expense-savings trend, and date-range totals.
Quality	2-second performance targets, validation, clear feedback, persistence, modular architecture and error handling.

The baseline also specifies starter categories Food, Rent, Transport, Bills, Entertainment and Other; custom categories remain supported. Budgets apply to the current month, and CSV imports reject unknown categories rather than creating them automatically.

3. Architecture & Design

ExpenseMate uses a local client-server arrangement: the browser acts as the client, Streamlit runs the application server, and SQLite provides the data layer. Internally, the design is separated into Presentation/UI, Services/Business Logic, and Data/Persistence layers, matching NFR-7. This separation keeps UI code independent from database operations and keeps calculations and validation reusable.

Layer	Main Components	Responsibility
Presentation	Streamlit pages, UI components, charts	Forms, tables, alerts, dashboards and visualization.
Services	Budget, transaction, category, savings, money, validators, CSV	Business rules, calculations, validation and orchestration.
Data	SQLite database + repositories	Persistence, CRUD operations, constraints and relational integrity.

The SRS defines use cases such as Add Transaction, Edit/Delete Transaction, Manage Categories, Set/View Budget Status, Import/Export CSV, View Budget Chart, View Income vs Expense, View Monthly Savings, View Financial Summary, and Persist Data to SQLite. Foreign-key relationships and atomic operations support data integrity.

4. Implementation Summary

Implementation followed the requirements and was verified through automated tests. Core services handle strict date and amount validation, budget calculations, category validation, CSV parsing, savings calculations, and transaction orchestration. Repository modules perform SQLite CRUD and budget/category persistence, while Streamlit pages provide the user-facing workflows. Matplotlib is used for budget and three-series trend charts.

- **Input rules reject invalid dates, negative/non-numeric amounts, malformed transaction fields and invalid CSV rows.**
- **CSV import enforces the exact column order date,category,amount,type and uses an all-or-nothing policy.**
- **Budget calculations count expenses toward spending; income does not reduce category spending.**
- **Database operations maintain persistence and foreign-key integrity across connections.**
- **Phase 5 added scoped CSV export (all/this month/custom), bulk transaction deletion with confirmation, and two corrective bug fixes.**

Phase 5 refactoring added five functions while reducing average cyclomatic complexity from 3.07 to 3.01. The maintainability index remained Grade A across modules, and eight additional regression tests increased the test total from 144 to 152.

5. Testing Summary

Testing progressed from unit testing to integration/system testing and finally maintenance regression testing. Phase 3 executed 144 tests with 144 passed, 0 failed and 97% code coverage. Phase 4 reported 38 integration tests and 36 system/performance tests, all passing. Performance verification measured approximately 0.02 seconds for rendering 500 transactions and approximately 0.002 seconds for CRUD updates under the documented test conditions.

Testing Stage	Result	Evidence
Unit & Coverage	144/144 passed; 97% coverage	Core logic, validators, repositories, UI components.
Integration	38/38 passed	Services ↔ repositories ↔ SQLite; constraints and transactions.
System	36/36 passed	Complete Streamlit flows, validation, charts and performance.
Maintenance	152/152 passed; 97% coverage	Phase 5 regression suite; zero reported regressions.

Phase 4 recorded eight defects and reported all eight corrected, with a 100% defect-resolution rate and no open issues at that phase. The test suite also verified requirement traceability for functional requirements and key NFRs, including performance, validation, CSV integrity, architecture separation and database error handling.

6. Maintenance & Evolution Summary

Phase 5 moved ExpenseMate from an integrated baseline into active maintenance and refactoring. The work was documented using the maintenance classification of corrective, adaptive and perfective changes, together with an evolution review based on Lehman's Laws of Software Evolution.

Type	Change	Purpose / Result
Corrective	BUG-09: incorrect over-budget status	Strict spent > limit check now aligns budget status with displayed figures.
Corrective	BUG-10: negative money formatting	Sign is handled before division so negative values display correctly.
Adaptive	Date-range CSV export	Exports can cover all transactions, current month or a custom range.
Perfective	Bulk transaction deletion	Multi-select checklist, Select All, preview and two-step confirmation improve workflow.
Regression	8 new automated tests	Protects the new fixes/features; total test count reached 152.

The Phase 5 metrics show controlled evolution: functions analyzed increased from 68 to 73, average cyclomatic complexity decreased from 3.07 to 3.01, Grade A functions increased from 60 to 65, and maintainability remained Grade A. The report's Lehman analysis relates the changes to continuing change, complexity control, self-regulation, organizational stability, familiarity, continuing growth, quality preservation and feedback-driven evolution.

7. Conclusion

ExpenseMate reached a tested and maintainable final baseline. The completed system covers the defined requirements for personal transaction tracking, categories, monthly budgeting, CSV portability, analytics and persistent storage. Its layered architecture separates UI, business logic and data access, making the code easier to test and extend.

8.Lessons Learned

- **Requirements should remain traceable from specification through implementation and testing; the project used FR/NFR-to-test mapping to verify coverage.**
- **Automated testing is valuable beyond finding coding mistakes: it also protects fixes and new features from regressions.**
- **Validation and data-integrity rules must be enforced at the appropriate service/data boundaries, especially for money, dates and CSV imports.**
- **Small, focused service functions can add functionality without necessarily increasing overall complexity; Phase 5 reduced average cyclomatic complexity.**
- **Maintenance is part of the software lifecycle: real bugs, usability friction and changing usage needs can drive corrective, adaptive and perfective work.**
- **A modular architecture made it possible to evolve exports and transaction deletion while preserving the existing system structure.**

Final Tests	Coverage	Avg. Complexity	Maintainability	Open Defects
152 / 152	97%	3.01 (A)	Grade A	0 reported

Overall, the final report consolidates the project's requirements, architecture, implementation, verification and maintenance work into one concise record. The result is a local personal budget manager with documented testing evidence and an established basis for future maintenance and evolution.